Overview

McreatiK Live Gallery puts wedding photos on guests' phones within seconds of the shot. Studios get a dashboard to run events. Couples get a private portal for album picks and original downloads. Each event has two photo sets: a Guest gallery (compressed, for guests) and a Client gallery (full quality, for the couple); events made before 11 Nov 2026 keep one set.

> **Note** This documentation was written from a read-only review of the `main` branch. Every statement points to a real file or function. Anything that could not be confirmed from the code is marked **(assumption)**. Secret values are never shown; settings are named, not printed.

What the product does

WHO	WHAT THEY DO	WHERE
Guests	Scan a QR code, watch photos arrive live, download and share	`/e/{slug}`
Studios	Create events, connect cameras, manage photos, give the couple a login	`/admin`
McreatiK	Everything a studio can, plus studios, plans, payments and gallery switches	`/admin` (super admin)
Couples	Pick album photos, download originals and ZIPs	`/client`
Camera operator	Runs the uploader app on a laptop at the venue	Desktop app

The three apps

APP	FOLDER	BUILT WITH	RUNS ON
Web app	`web/`	React 19, TypeScript, Vite, Tailwind CSS 4, react-router	Cloudflare Workers static assets: live.mcreatik.com
Backend API	`backend/`	Spring Boot 4.1, Java 21, JPA + JdbcTemplate, Flyway	Docker on the Oracle VM: im-api.mcreatik.com
Uploader	`uploader/`	Java Swing, SQLite queue, built-in FTP server	The photographer's laptop

They share one PostgreSQL database (Supabase) and two Cloudflare R2 buckets. Originals sit in a private bucket. Gallery copies sit in a public bucket served at im-media.mcreatik.com.

How to read these docs

Running the service day to day? Start with **SOPs** (one per role: McreatiK admin, Studios, Couples, Guests) and the **Camera & uploader guides** (Uploader, Canon, Sony, Event-day checklist). The rest is the technical reference:

1. **Architecture** shows how the parts connect.
2. **Frontend**, **Backend**, **Database** and **API reference** explain the code.
3. **User journeys** follow each person through the system.
4. **Security and performance** and **Deployment** cover running it.
5. **Findings and next actions** lists every issue found, ranked.
6. **Glossary** explains the technical words.

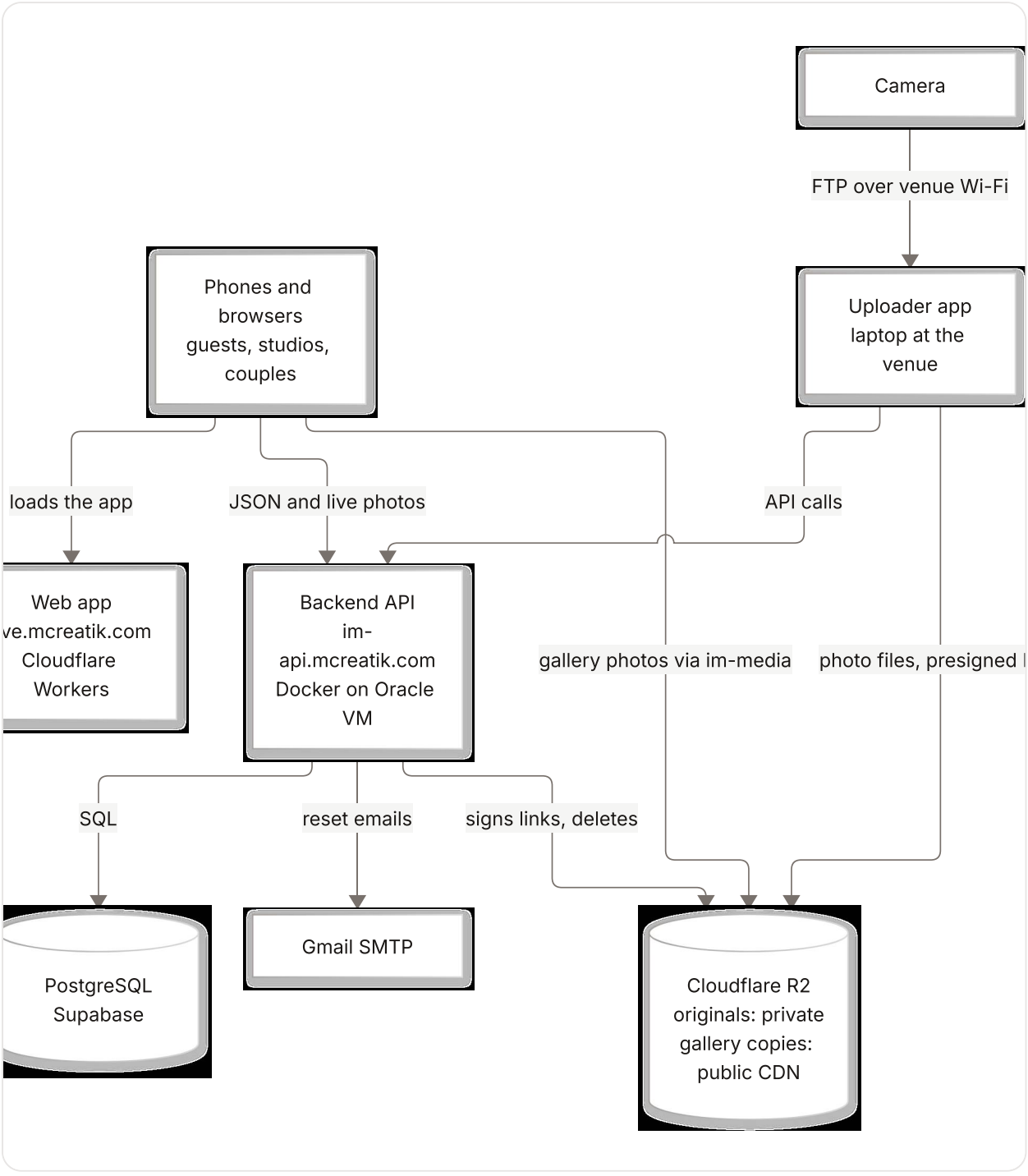
Repository map

```text
mcreatik-instant-memories/
├── backend/          Spring Boot API, Flyway migrations, these docs (src/main
├── web/              React app: guest gallery, studio dashboard, couple porta
├── uploader/         Desktop uploader for the venue laptop
├── deploy/oracle/    deploy.sh, rollback.sh, nginx and Caddy configs, env exa
└── .github/          CI, backend image build, Dependabot
```

Architecture

Photo files never pass through the backend server. Uploads go straight to Cloudflare R2 through short-lived signed links, and guests load photos from the im-media CDN. The backend signs links, enforces rules, keeps records and pushes live updates. That is why a 1-CPU server can run a busy wedding.

System map


Camera
FTP over venue Wi-Fi
Phones and browsers guests, studios, couples
Uploader app laptop at the venue
loads the app
JSON and live photos
API calls
Web app ve.mcreatik.com Cloudflare Workers
Backend API im-api.mcreatik.com Docker on Oracle VM
gallery photos via im-media
photo files, presigned
SQL
reset emails
signs links, deletes
PostgreSQL Supabase
Gmail SMTP
Cloudflare R2 originals: private gallery copies: public CDN

How the parts talk

CHANNEL	USED FOR	CODE
JSON over HTTPS with a bearer token	Everything the dashboard, portal and uploader read or change	`web/src/lib/api.ts`, `uploader/.../ApiClient.java`
Presigned URLs (signed, short-lived links)	Uploading and downloading files directly to and from R2	`storage/R2StorageService`
Server-Sent Events (SSE)	Pushing new photos to guests' phones	`realtime/SseGalleryBroadcaster`, `gallery/useLiveGallery.ts`
CDN	Serving gallery and thumbnail images	R2 media bucket at im-media.mcreatik.com

Upload paths

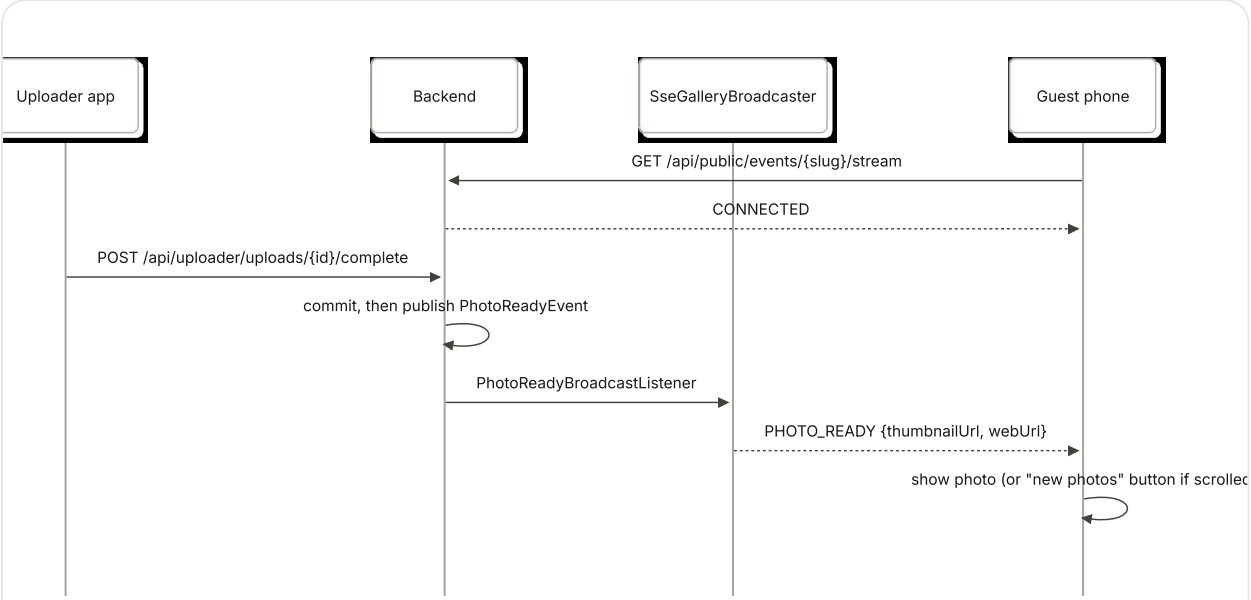
There are three ways a photo enters the system:

PATH	WHO	WHAT HAPPENS
Camera uploader, prepared	Uploader app (default)	Laptop makes 2048 px and 640 px JPEGs, PUTs them to R2 media, photo is READY at once
Camera uploader, original only	Uploader app (fallback)	Original goes to R2 originals; the server's worker makes the versions
Dashboard	Studio in the browser	Browser resizes (`prepareUpload.ts`) and posts both versions; originals optional via presigned PUT

Guest gallery and Client gallery

Every photo has a `gallery` value, `GUEST` or `CLIENT` (`photos.gallery`), and so does every category. Camera uploads and compressed dashboard uploads are `GUEST`. Full-quality finals uploaded by the studio are `CLIENT`. Guests are served only `GUEST` photos. On events with `split_galleries = true` the couple's portal, album picks and ZIPs use only `CLIENT` photos; older events keep showing the couple every photo. Uploads are stored even while a gallery is switched off. See the Database and API reference pages.

Live updates



Events sent to guests: `PHOTO_READY`, `PHOTO_REMOVED`, `EVENT_STATUS`, `GALLERY_CLOSED`. On reconnect, the phone fetches anything it missed with `?after=<cursor>`, and it repeats that as a safety poll every 60 seconds.

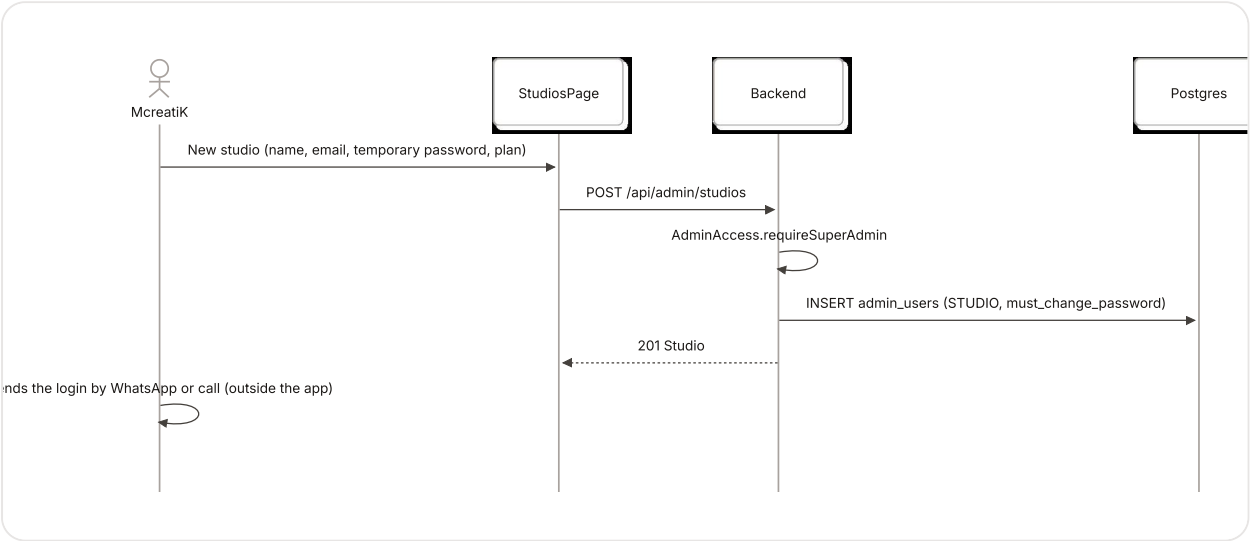
Key design decisions

DECISION	WHY
Direct-to-R2 uploads and CDN delivery	The 900 MB / 1 CPU container never carries image bytes
Photos table doubles as the job queue	No extra queue service; jobs can't be lost between a queue and the database
Stateless JWT logins	No server sessions; any request can be checked on its own
Prepare photos on the laptop	About 1 MB per photo on weak venue internet instead of 10 to 60 MB
Lazy-loaded dashboard code	Guests download only the gallery
One Docker container, in-memory rate limits and SSE	Simple and cheap; the trade-off is a single instance (see Security)

User journeys

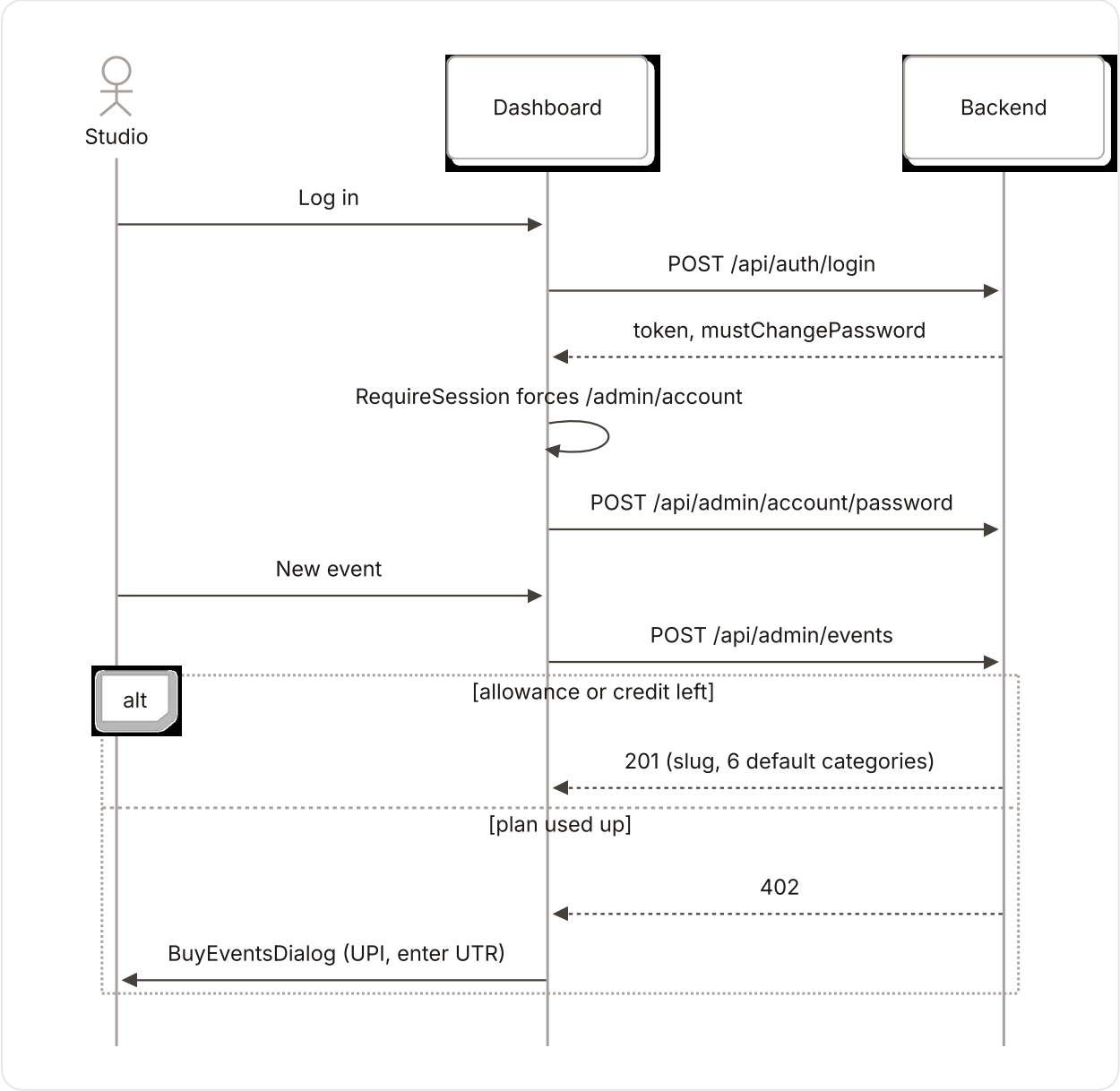
Each journey follows one person from the screen, through the API and backend, to the table or file storage it touches.

McreatiK onboards a studio



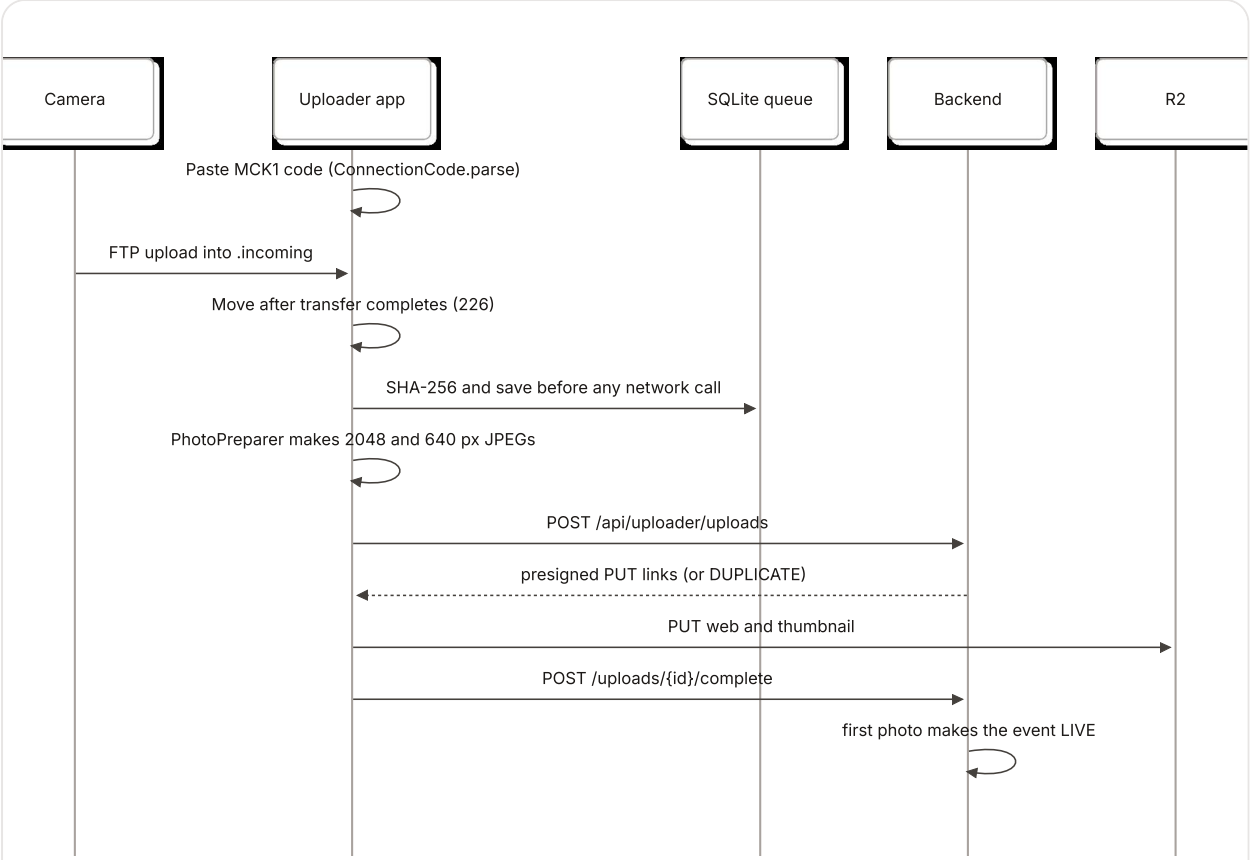
1. `AdminBootstrap` creates McreatiK's own account on first boot.
2. `StudioAccordionItem` (one row per studio on `StudiosPage`) holds the plan, monthly quota, credits and `BrandingCard`, with Reset password, Disable and Delete.
3. `PricingPage` (menu: Pricing & payments) holds the prices and `PaymentsToCheck`, which lists PENDING UPI payments. McreatiK checks the UTR in the bank app, then approves, which adds credits.
4. `GalleryAccessCard` turns an event's guest or couple gallery off. Open phones get `GALLERY_CLOSED` after the commit.

A studio runs an event



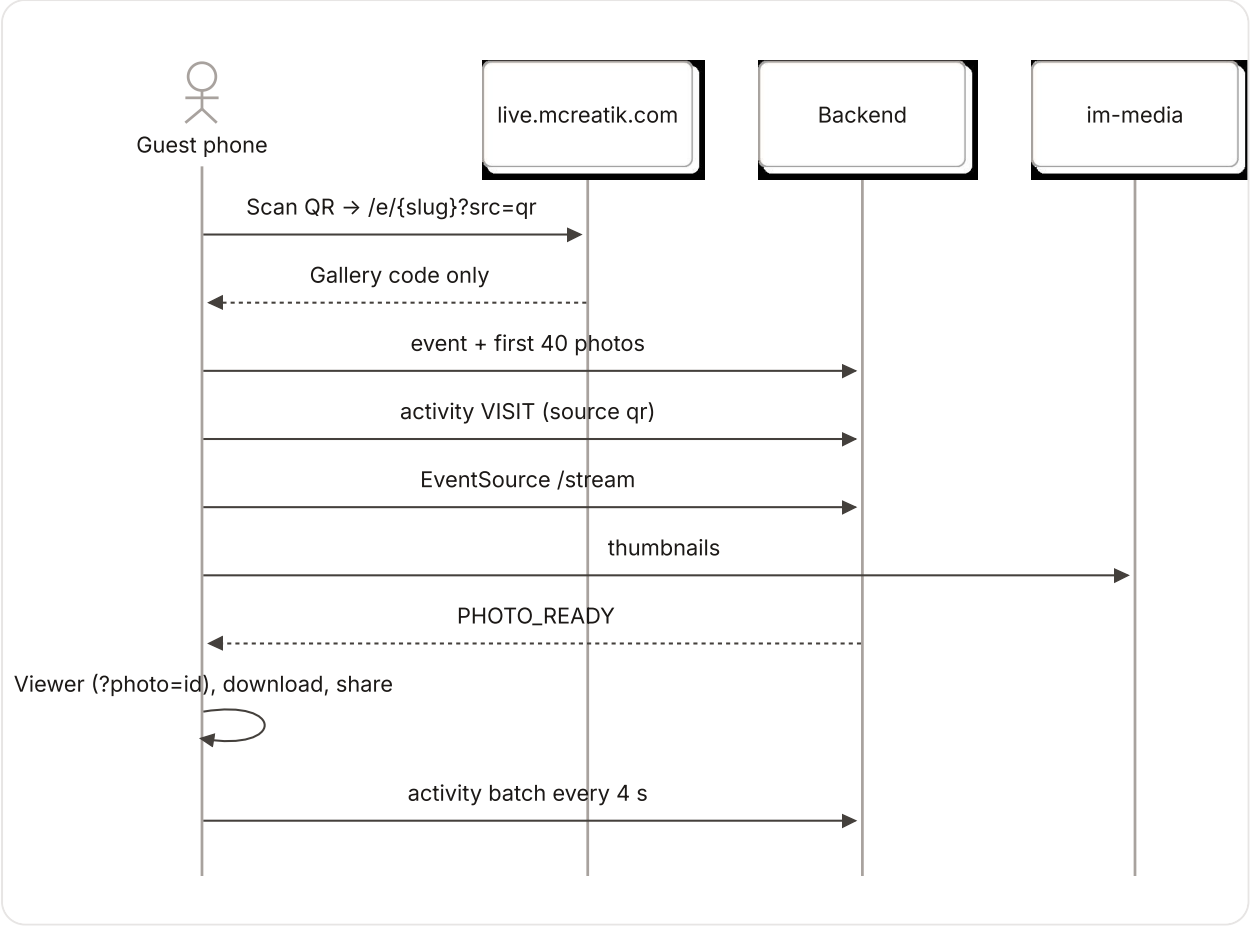
1. **Prepare.** Print the QR from `QrPanel` or `PrintQrPage`; the link carries `?src=qr` to count scans. Add an uploader in `UploadersPanel` and copy the `MCK1.` connection code, which is shown once.
2. **During the event.** `PhotosPanel` likes, hides, sets the cover, deletes and moves photos. `AnalyticsPanel` refreshes every 30 s.
3. **Afterwards.** Set the status to COMPLETED. Create the couple's login in `ClientsCard` and send it with `SendButtons`. Download the album picks as a ZIP.

The camera operator at the venue



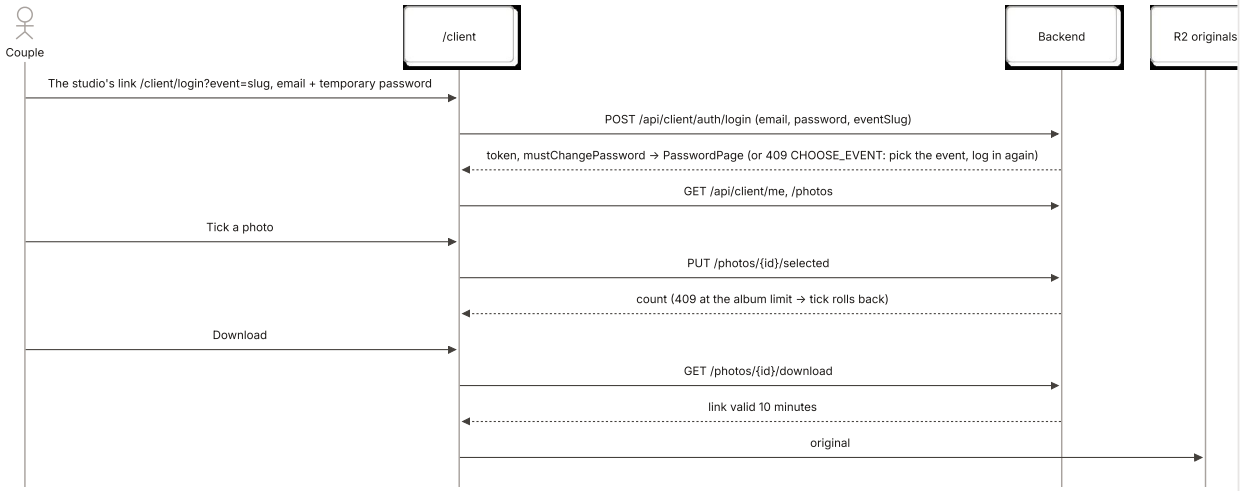
- **Connection code:** one string, `MCK1.<base64 {server, token}>`, so nobody types a URL and a 48-character token at the venue.
- **No half photos:** `FtpPhotoSource` moves a file into the watched folder only after a complete transfer.
- **Nothing lost:** `UploadEngine` queues first. Network errors retry with backoff. A revoked token or archived event pauses uploading. A restart returns in-flight photos to the queue.
- **Online status:** the dashboard shows the uploader online if a heartbeat arrived in the last 45 s, with its queue counts.

A guest at the wedding



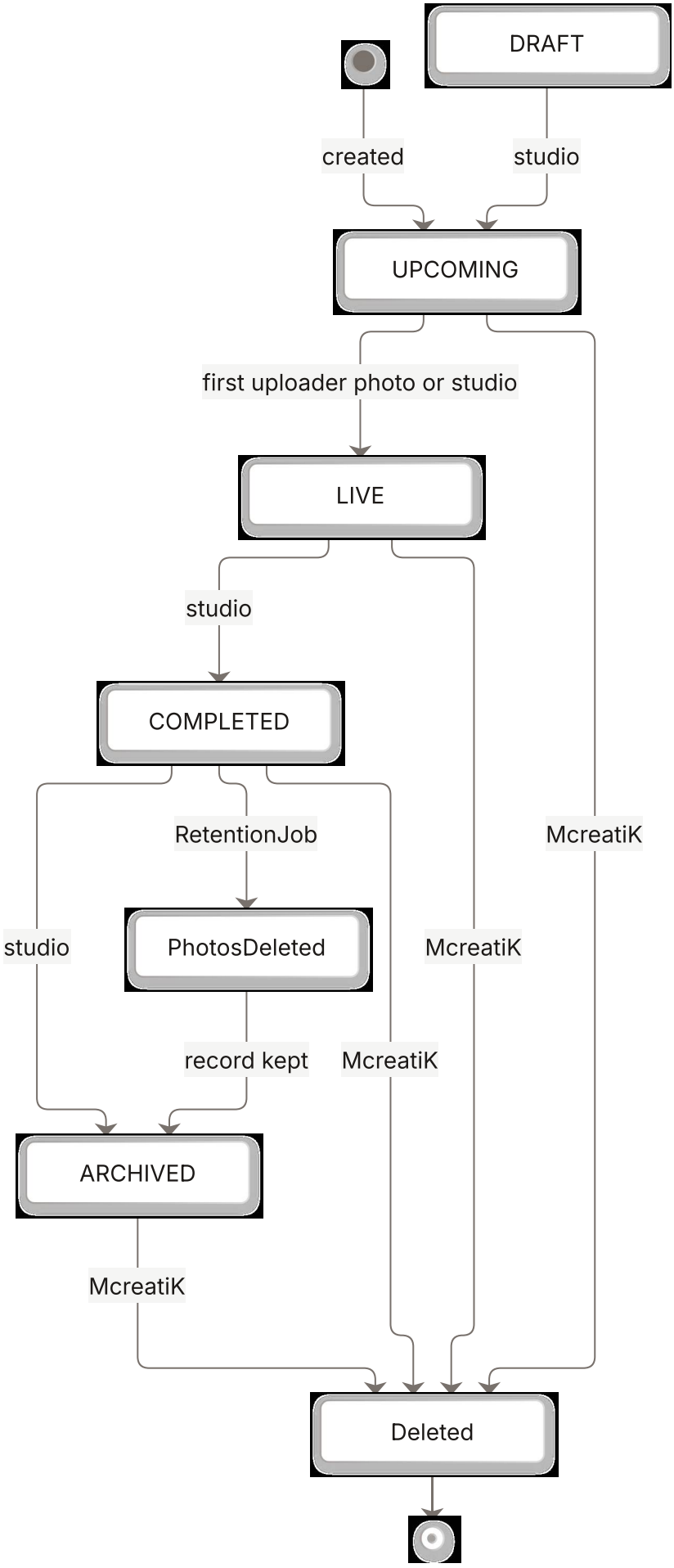
- **No sign-up.** The browser keeps only a random visitor ID.
- **Possible screens:**
  - not found (draft or archived event)
  - unavailable (McreatiK switched the gallery off)
  - the live grid
- **No jumping grid.** New photos wait behind a "new photos" button when the guest has scrolled down or has a photo open.
- **Removal requests.** Every photo has a "Request removal" email link to connect@mcreatik.com.
- **No originals.** Guests only get gallery copies.

The couple



Forgot password emails only while the login is active, the event isn't archived and the couple's gallery is on.

Event life cycle


DRAFT
created
studio
UPCOMING
first uploader photo or studio
LIVE
studio
COMPLETED
RetentionJob
McreatiK
studio
PhotosDeleted
McreatiK
record kept
McreatiK
ARCHIVED
McreatiK
Deleted

- Guests can see UPCOMING, LIVE and COMPLETED events (`EventStatus.isPubliclyVisible`).
- Every status except ARCHIVED accepts uploads.

> **Note** The automatic switch to LIVE (`promoteToLive`) does not send `EVENT_STATUS`. A guest who opened the gallery early keeps the "Upcoming" badge until they reload, even while photos arrive.
